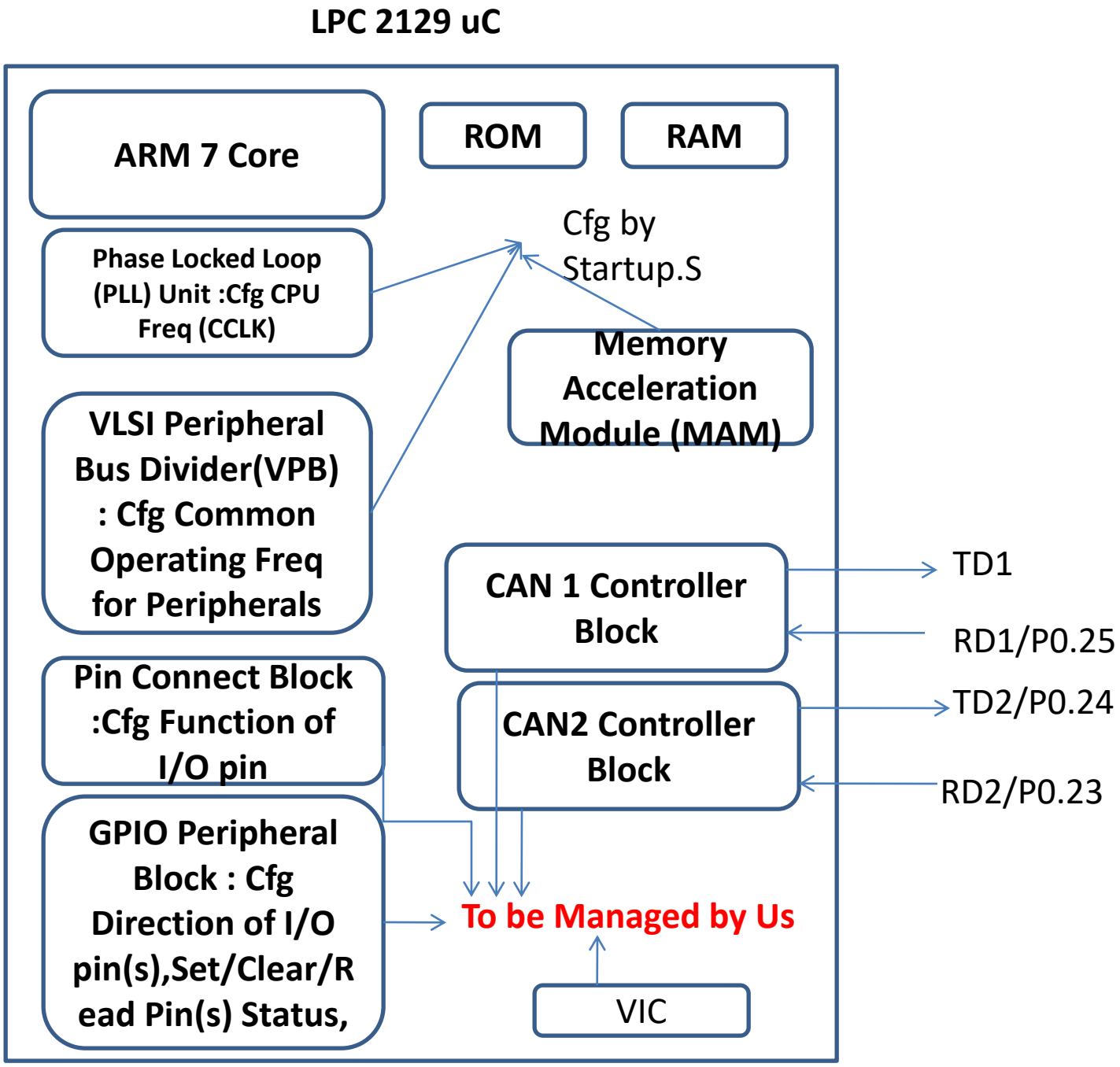
CAN PROTOCOL IMPLEMENTATION (LOW LEVEL CAN DRIVER/FIRMWARE) USING LPC2129 uC & Keil Embedded C for ARM

LEVEL 1


LPC 2129 uC
ARM 7 Core
ROM
RAM
Phase Locked Loop (PLL) Unit :Cfg CPU Freq (CCLK)
Cfg by Startup.S
Memory Acceleration Module (MAM)
VLSI Peripheral Bus Divider(VPB) : Cfg Common Operating Freq for Peripherals
CAN 1 Controller Block
TD1
RD1/P0.25
Pin Connect Block :Cfg Function of I/O pin
CAN2 Controller Block
TD2/P0.24
RD2/P0.23
GPIO Peripheral Block : Cfg Direction of I/O pin(s),Set/Clear/Read Pin(s) Status,
To be Managed by Us
VIC

CALCULATIONS FOR CAN: SPREAD SHEET FOR QUANTA vs BRP

@PCLK 60MHz,&

@BIT_RATE 125KHz,

BRP=PCLK/(BIT_RATE*QUANTA)

BRP=(480/QUANTA) ,where 8>= QUANTA <=25

QUANTA = 8, BRP = 60

QUANTA = 10,BRP = 48

QUANTA = 12,BRP = 40

QUANTA = 15,BRP = 32

QUANTA = 16,BRP = 30

QUANTA = 20,BRP = 24

QUANTA = 24,BRP = 20

SPREAD SHEET FOR QUANTA vs BRP

@PCLK 12MHz,&

@BIT_RATE 125KHz,

BRP=PCLK/BIT_RATE*QUANTA

BRP=(96/QUANTA) ,where 8>= QUANTA <=25

QUANTA = 8, BRP = 12

QUANTA = 16,BRP = 6

QUANTA = 24,BRP = 4

Calculations For CAN

Since SAMPLE_POINT (70%) = 0.7 * QUANTA

= 0.7 * 16

= 11.2 (ignore decimal part)

= 11 (1+TSEG1)

Since NBT=1+(PropSeg+PS1)+PS2=1+TSEG1+TSEG2

TSEG1 = 10

TSEG2 = 5

if(Tseg2 >= 5 Tq)

SJW=4 Tq

if(Tseg2 < 5 Tq)

SJW =(Tseg2 - 1) Tq

CAN Controller SFRs in LPC2129 uC AtLeast

CAN 1 Controller SFRs

- C1MOD
- AMFR
- C1BTR
- C1GSR
- C1CMR
- C1TID1
- C1TFI1
- C1TDA1
- C1TDB1
- C1RID
- C1RFS
- C1RDA
- C1RDB

CAN 2 Controller SFRs

- C2MOD
- AMFR
- C2BTR
- C2GSR
- C2CMR
- C2TID1
- C2TFI1
- C2TDA1
- C2TDB1
- C2RID
- C2RFS
- C2RDA
- C2RDB

```
/* can.h */
#ifndef __CAN_H__
#define __CAN_H__

#include "types.h"

struct CAN_Frame
{
    u32 ID,RTR DLC,Data1,Data2;
};

void Init_CAN1(void);
void CAN1_Tx(struct CAN_Frame);
void CAN1_Rx(struct CAN_Frame *);

#endif
```

Understanding Code Block for Initialization of

CAN Controller 1

How BTR_LVAL is Computed in can_defines.h

- //defines required for C1BTR in can_defines.h
- #define PCLK 60000000 //Hz
- #define BIT_RATE 125000 //Hz
- #define QUANTA 16
- #define BRP (PCLK/(BIT_RATE*QUANTA))
- #define SAMPLE_POINT (0.7 * QUANTA)
- #define TSEG1 ((int)SAMPLE_POINT-1)
- #define TSEG2 (QUANTA-(1+TSEG1))
- #define SJW ((TSEG2 >= 5) ? 4 : (TSEG2-1))
- #define SAM 0 //0 or 1 ,sample bus 1 or 3 time(s)
- **#define BTR_LVAL (SAM<<23|(TSEG2-1)<<20|(TSEG1-1)<<16|(SJW-1)<<14|(BRP-1))**
    - //defines for C1CMR bit set
        - #define TR_BIT 0
        - #define RRB_BIT 2
        - #define STB1_BIT 5
    - //defines for C1GSR bit check
        - #define RBS_BIT 0
        - #define TBS1_BIT 2
        - #define TCS1_BIT 3

CAN Controller 1 Initialization : (defined in can.c)

```
void Init_CAN1(void)
{
   //cfg p0.25 pin as CAN1_RX pin(RD1),TD1 is exclusive
   CFGPIN(PINSEL1,(25-16),RD1);
   //Reset CAN1 controller using C1MOD
  //cfg AFMR for all received messages are accepting
  //Set baud Rate for CAN using C1BTR
  //Enable CAN1 controller
}
```

Understanding Code Block for Transmission of

CAN Frame

```
                    //  defined in can.c
void CAN1_Tx(struct CAN_Frame txFrame)
{
  // Checking that the TX buffer is empty in C1GSR
   // place 11-bit tx id in C1T1D1
   // cfg whether data/remote frame & no of data bytes in message
   // For Data Frame place 1 to 8 bytes data into Data Tx Buffers
   //Place data bytes 1-4 in C1TDA1
   //Place data bytes 5-8 in C1TDB1
  //Select Tx Buf1 & Start Xmission using
  //monitor tx status in C1GSR
}
```

Now
Using Keil IDE
To

Simulate CAN Communication-Transmission

Understanding Code Block for Reception of

CAN Frame

```
//defined in can.c
void CAN1_Rx(struct CAN_Frame *rxFrame)
{
   //wait for CAN frame recv status
   //read 11-bit CANid of recvd frame.
   // read & extract data/remote frame status
   //read & extract data length
   //check if recvd frame is data frame,extract data bytes
    //read 1-4 bytes from C1RDA
    //read 5-8 bytes from C1RDB
    // Release receive buffer
}
```

Understanding Code Block for Debug Script for

CAN 1 Peripheral

/* can_debug.ini */

/* Note:

A debug script for CAN debugging & simulation using Keil IDE.

Can be used for representing a CAN NODE connected to the bus which sends

CAN(Standard Data/Remote) frame .

*/

Now
Using Keil IDE
To

Simulate CAN Communication-Reception